

Problem Set 1

Peter Shi, Wenhui Xu

June 21, 2026

Question 1: Rounding Error

Using the well-known formula for the roots of a quadratic equation $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ compute the larger root of the following quadratic equation: $ax^2 + bx + c = 0$ for the following values: $a = 1$, $b = 100,000$, and $c = 10^n$, $n = -1, -2, \dots, -8$. Now use the following alternative method. First calculate $q = -\frac{1}{2} \left(b + (b) \sqrt{b^2 - 4ac} \right)$, and then obtain the two roots: $x_1 = q/a$ and $x_2 = c/q$. Notice that the two calculations should give you precisely the same answer if it were not for the rounding error of the computer arithmetic. Also note that in some real life computational problems you will need precision up to the 10th or 12th significant digit (for example, to find the equilibrium price that clears an asset market). How close are the answers for different values of n ?

We solve the quadratic equation $ax^2 + bx + c = 0$ with $a = 1$, $b = 100000$, and $c = 10^n$ for $n = -1, -2, \dots, -8$. The goal is to compute the **larger root** and compare the results across different methods.

Method 1: Usual quadratic formula

Compute the larger root using

$$r_{\text{old } 1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

In this problem, b is very large and $\sqrt{b^2 - 4ac} \approx b$, so the subtraction $-b + \sqrt{\cdot}$ suffers from **catastrophic cancellation** (subtracting two nearly equal large numbers). This can destroy significant digits in floating-point arithmetic.

```

using Pkg
Pkg.add("PrettyTables")
Pkg.add("DataFrames")
using PrettyTables
using DataFrames

#Instead of writing iteration loops, we contrusted a function taking vectors for
    butch computation.

function quadratic_roots(a::AbstractVector, b::AbstractVector, c::AbstractVector
    )
    @assert length(a) == length(b) == length(c) #ensuring elementwise operations
        are well-defined / broadcast safely

    Δ = b.^2 .- 4 .* a .* c #broadcasting operation to be per element
    r1 = (-b .+ sqrt.(Δ)) ./ (2 .* a) #larger root result(with rounding errors)
    r2 = (-b .- sqrt.(Δ)) ./ (2 .* a)

    return r1, r2
end

```

Method 2: Alternative Method

$$q = -\frac{1}{2} \left(b + \text{sign}(b) \sqrt{b^2 - 4ac} \right), \quad r1_{new} = \frac{q}{a}, \quad r2_{new} = \frac{c}{q}.$$

Since $b > 0$ here, $\text{sign}(b) = 1$ and q is formed by adding two numbers of the same sign (avoiding cancellation). The **larger root** is then taken as $x_{\text{larger}} = \frac{c}{q}$.

```

#Similarly, for constructing the alternative method function call, we utilize
    vectors to achieve batch computation.

function newQuadraticRoots(a::AbstractVector, b::AbstractVector, c::
    AbstractVector)
    @assert length(a) == length(b) == length(c)

    Δ = b.^2 .- 4 .* a .* c #ensuring elementwise operations are well-defined /
        broadcast safely

    q = -0.5 .* (b .+ sign.(b) .* sqrt.(Δ))

    r1 = c ./ q #larger root result(without rounding errors)
    r2 = q ./ a

    return r1, r2
end

```

Result Comparison

n <i>Int64</i>	c <i>Float64</i>	r_large_error <i>Float64</i>	r_large_precise <i>Float64</i>	err_absdiff <i>Float64</i>	r2_old <i>Float64</i>	r2_new <i>Float64</i>	absdiff <i>Float64</i>
1	0.1	-9.99999e-7	-1.0e-6	7.00957e-13	-14285.7	-14285.7	0.0
2	0.01	-1.00001e-7	-1.0e-7	7.61449e-13	-14285.7	-14285.7	0.0
3	-0.001	1.00003e-8	1.0e-8	2.84029e-13	-14285.7	-14285.7	0.0
4	0.0001	-9.99924e-10	-1.0e-9	7.55393e-14	-14285.7	-14285.7	0.0
5	1.0e-5	-9.97846e-11	-1.0e-10	2.15438e-13	-14285.7	-14285.7	0.0
6	1.0e-6	-1.03942e-11	-1.0e-11	3.94225e-13	-14285.7	-14285.7	0.0
7	1.0e-7	-1.03942e-12	-1.0e-12	3.94225e-14	-14285.7	-14285.7	0.0
8	1.0e-8	0.0	-1.0e-13	1.0e-13	-14285.7	-14285.7	0.0

The table below shows how the error changes with n . In particular, as n decreases (so c becomes smaller), the usual quadratic formula has a similar magnitude of errors due to cancellation, while the alternative method remains well-behaved.

```
#Initializing vectors and match the dimensions

a = fill(7.0, 8)
b = fill(100000.0, 8)

c = [10^-1.0, 10^-2.0, -10^-3.0, 10^-4.0, 10^-5.0, 10^-6.0, 10^-7.0, 10^-8.0]

r_large_precise, r2_new = newQuadraticRoots(a, b, c) #Unpack the function output
(r1, r2) into two separate variables.
err_absdiff = abs.(r_large_error .- r_large_precise)
r2_absdiff = abs.(r2_old .- r2_new) #The smaller root shows no sign of rounding
errors

using Printf

println(" n          c          r_large_error          r_large_precise
      err_absdiff  r2_old    r2_new    absdiff")
for (i, n_val) in enumerate(1:8)
    @printf("%2d %10.1e % .16e % .16e % .3e % .3e % .3e %6.2f\n",
           n_val, c[i], r_large_error[i], r_large_precise[i], err_absdiff[i],
           r2_old[i], r2_new[i], r2_absdiff[i])
end

n_vals = collect(1:8)

data = DataFrame(
    n = n_vals,
    c = c,
    r_large_error = r_large_error,
    r_large_precise = r_large_precise,
    err_absdiff = err_absdiff,
    r2_old = r2_old,
    r2_new = r2_new,
    absdiff = r2_absdiff
)
```

```
pretty_table(stdout, data, backend=:latex)
```

Question 2: Truncation Error

Truncation error/Unstable Algorithms: Consider the “Golden Mean” which is the number given by:

$$\phi = \frac{\sqrt{5} - 1}{2} \approx 0.61803398.$$

- (a) First, verify analytically that the powers of this number satisfy the following recursion (Hint: trivial!)

$$\phi^{n+1} = \phi^{n-1} - \phi^n.$$

This recursion suggests a seemingly efficient way to compute higher powers of ϕ since it only requires subtraction which is much faster than multiplication (required to exponentiate ϕ directly).

- (b) But how well does the clever algorithm described in (a) work in practice? To see this, compute ϕ^n for $n = 2, 3, \dots, 20$ using two separate methods. First, write a simple program using the recursion above and the fact that $\phi^0 = 1$ and $\phi^1 = 0.61803398$. Second, simply raise ϕ to the appropriate power. Compare the values obtained using the two methods. How do they compare to each other as n increases? This (somewhat contrived) exercise illustrates the dangers of rounding and truncation errors that combine together to create a dirty mess.

a) We know that Φ is identified as the solution to the equation $\Phi^2 + \Phi = 1$, $\Phi^2 = 1 - \Phi$. It then follows that if both sides were multiplied by Φ^{n-1} that $\Phi^{n+1} = \Phi^{n-1} - \Phi^n$

```
Phi = (sqrt(5.0) - 1.0)/2.0
function goldenRecursion(n::Int)

Phi = (sqrt(5.0) - 1.0)/2.0

if n == 0
    return 1.0

elseif n == 1
    return Phi

else
    return goldenRecursion(n-2) - goldenRecursion(n-1)

end
end
goldenRecursion.(collect(2:1:20))
Phi.^ collect(2:1:20)
n_range = collect(2:1:20)
powers_phi_recursion = goldenRecursion.(n_range)
powers_phi_analytical = Phi.^ n_range
abs_diff_phi = abs.(powers_phi_recursion .- powers_phi_analytical)
```

b We see that the absolute difference in the recursive vs analytical powers of Φ increases as n increases.

Table 1: Comparison of Phi powers (Recursive vs. Analytical)

n <i>Int64</i>	Recursive_Phi <i>Float64</i>	Analytical_Phi <i>Float64</i>	Absolute_Difference <i>Float64</i>
2	0.381966	0.381966	1.11022e-16
3	0.236068	0.236068	5.55112e-17
4	0.145898	0.145898	2.22045e-16
5	0.0901699	0.0901699	2.35922e-16
6	0.0557281	0.0557281	4.64906e-16
7	0.0344419	0.0344419	6.8695e-16
8	0.0212862	0.0212862	1.15533e-15
9	0.0131556	0.0131556	1.83707e-15
10	0.00813062	0.00813062	2.99413e-15
11	0.005025	0.005025	4.82947e-15
12	0.00310562	0.00310562	7.82534e-15
13	0.00191938	0.00191938	1.26546e-14
14	0.00118624	0.00118624	2.04806e-14
15	0.000733137	0.000733137	3.3135e-14
16	0.000453104	0.000453104	5.36156e-14
17	0.000280034	0.000280034	8.67505e-14
18	0.00017307	0.00017307	1.40366e-13
19	0.000106963	0.000106963	2.27117e-13
20	6.6107e-5	6.6107e-5	3.67483e-13

```

phi_data = DataFrame(
    n = n_range,
    Recursive_Phi = powers_phi_recursion,
    Analytical_Phi = powers_phi_analytical,
    Absolute_Difference = abs_diff_phi
)

println("Comparison of Phi powers (Recursive vs. Analytical):")
pretty_table(stdout, phi_data, backend=:latex)

```

Question 3: VFI

a). Using the standard VFI, obtain numerical solution, both the value function and the savings rule.

- i. How many iterations it took for VFI to stop?
- ii. Plot both functions and make sure to choose the scale of your graph and other details so that the figure is informative (not too squeezed, etc.). Comment on the shape of both functions and their properties.

i) Using $N = 1001$, $tol = 10^{-8}$ we found that the VFI algorithm converged after 57 iterations

```
u(c) = (c > 0) ? log(c) : -Inf #define log utility

f(k; A=1.0, alpha=0.4, delta=0.1) = A * k^alpha + (1.0 - delta) * k #define
    production function

function kstar_and_grid(; beta=0.95, A=1.0, alpha=0.4, delta=0.1,
    k_low=0.25, k_high=1.75, N=11)

    # steady-state exists if  $(1/\beta - (1-\delta)) > 0$ 
    rhs = (1/beta - (1 - delta)) / (alpha * A)
    @assert rhs > 0 "Steady-state formula requires  $(1/\beta - (1-\delta)) / (\alpha A) > 0$ ."

    kstar = rhs^(1/(alpha - 1))

    kmin = k_low * kstar
    kmax = k_high * kstar
    @assert kmin > 0 && kmax > kmin

    kgrid = range(kmin, kmax; length=N)

    return kstar, kmin, kmax, kgrid
end

function standard_vfi(;beta=0.95, A=1.0, alpha=0.4,delta=0.1,N=11,
    k_low=0.25, k_high=1.75,tol=1e-8, maxit=500000)

    # Step 0: grid (make sure it uses the same parameters)
    kstar, kmin, kmax, kgrid = kstar_and_grid(;beta=beta, A=A, alpha=alpha,delta
        =delta,k_low=k_low, k_high=k_high, N=N)
    @assert length(kgrid) == N
    @show first(kgrid) last(kgrid) step(kgrid)
    # Step 1: initial guess V0
    V = zeros(Float64, N) #vector length of 11, the up-to-date value functions
    policy_idx = fill(1, N) #the up-to-date policy functions
    j0 = argmin(abs.(kgrid .- kstar)) # index of grid point closest to kstar
    policy_idx .= j0
```

```

policy_idx_new = similar(policy_idx) # the current policy functions
V_new = similar(V) #vector length of 11 the current value functions

it = 0
converged = false

while it < maxit

    it += 1

    # Step 2(i): maximization step
    dist = 0.0
    for i in 1:N
        k = kgrid[i]

        best_val = -Inf # Initialized to -Inf to correctly capture negative
            utility values
        best_j = 1

        for j in 1:N
            kp = kgrid[j]
            c = f(k, A=A, alpha=alpha, delta=delta) - kp
            val = u(c) + beta * V[j] # u(c) should be -Inf if c<=0

            if val > best_val
                best_val = val
                best_j = j
                #println(val)
                #println(best_val)
            end
        end

        policy_idx_new[i] = best_j

        # Step 2(ii): evaluation step, for each i, update once
        V_new[i] = best_val

        # Step 3: stopping rule check, for each i, update once
        dist = max(dist, abs(kgrid[policy_idx_new[i]] - kgrid[policy_idx[i]]
            ))
        #println("dist is ", dist)
    end

    # update for next iteration
    V.= V_new
    policy_idx .= policy_idx_new

    if dist <= tol
        println("STOP at it=$it with dist=$dist")
        converged = true
        break
    end
end

if it == maxit
    converged = false
end

if it % 50 == 0

```

```

        println("it=$it, dist=$dist, policy_idx[1:5]=", policy_idx[1:5])
    end

    g = kgrid[policy_idx]
    return (; V, g, kgrid, policy_idx, it, converged, kstar, kmin, kmax)
end

out = standard_vfi(N=1001, k_low=0.02, k_high=2.0)
@show out.it out.kstar out.kmin out.kmax out.g out.converged out.V

```

ii) We see that the graph for $V(k)$ is roughly logarithmic while the graph for $g(k)$ is roughly linear. This is in line with our expectations as the lifetime value of starting with more capital is expected to be increasing, but the marginal value of additional capital is expected to be decreasing for $V(k)$. For $g(k)$ linearity implies a constant proportion of existing production being invested into capital for the next term.

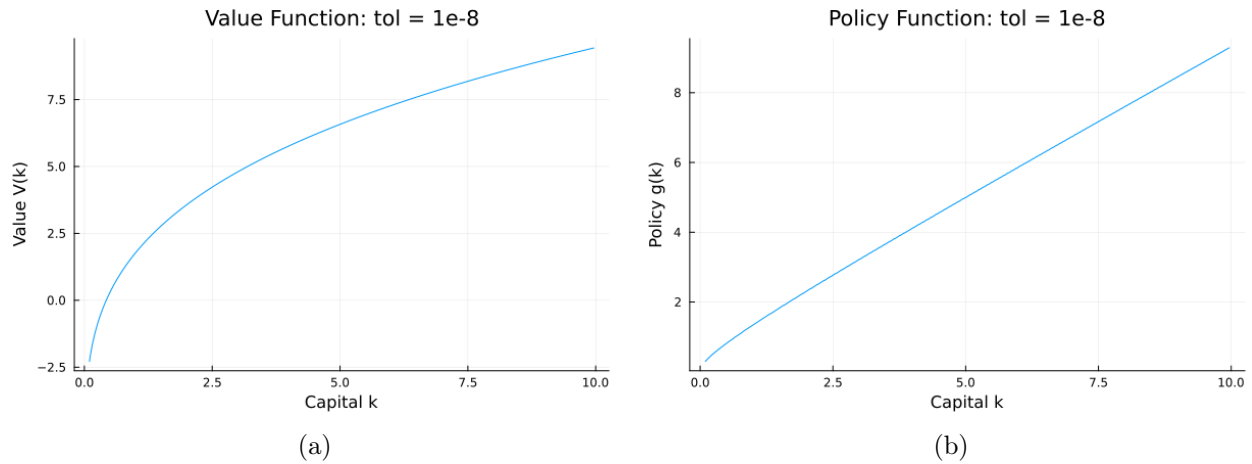


Figure 1: $V(k)$ and $g(k)$ with a tolerance of 10^{-8}

```
using Plots
plot(
    out.kgrid,
    out.V,
    xlabel = "Capital k",
    ylabel = "Value  $V(k)$ ",
    title = "Value Function: tol = 1e-8",
    legend = false,
    grid = true
)

plot(
    out.kgrid,
    out.g,
    xlabel = "Capital k",
    ylabel = "Policy  $g(k)$ ",
    title = "Policy Function: tol = 1e-8",
    legend = false,
    grid = true
)
```

b) Repeat part (a) by changing the stopping tolerance to 10^{-6} and to 10^{-5} . Answer the same questions as in (a).i and (a).ii. [Hint: *When you complete this part, it may look like there is something strange with part (b) – that is intentional. Comment on what you find.*]

i) Using $N = 1001$, $tol = 10^{-6}$ and 10^{-5} we found that the VFI algorithm converged after 57 iterations in both scenarios. Changing the tolerance did not change the number of iterations, this is because our k grid only had a minimum gap of $\sim 4 \times 10^{-4}$ which is well above the tested tolerance limits.

ii) We see the same shapes for $V(k)$ and $g(k)$ for the same reasons as aforementioned

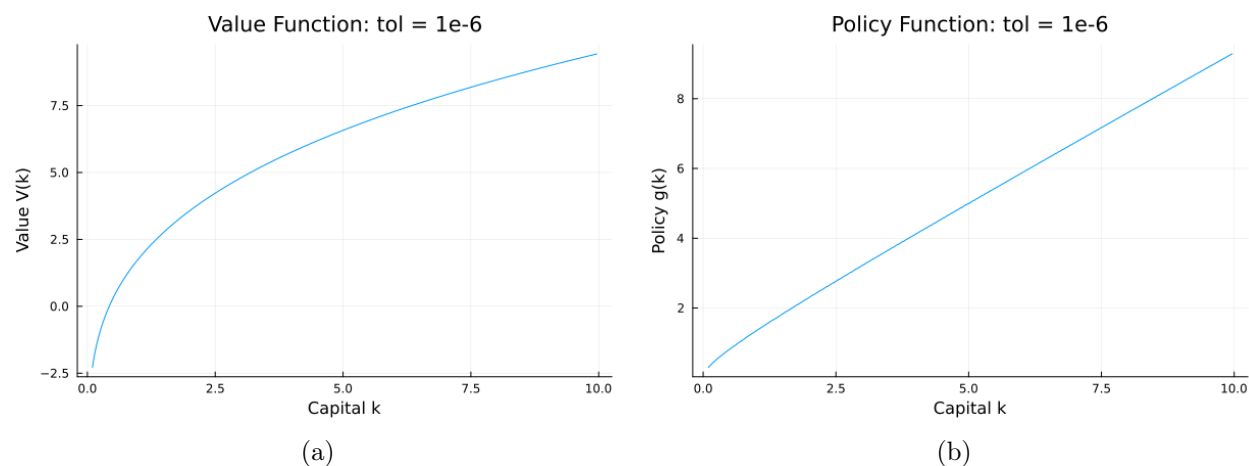


Figure 2: $V(k)$ and $g(k)$ with a tolerance of 10^{-6}

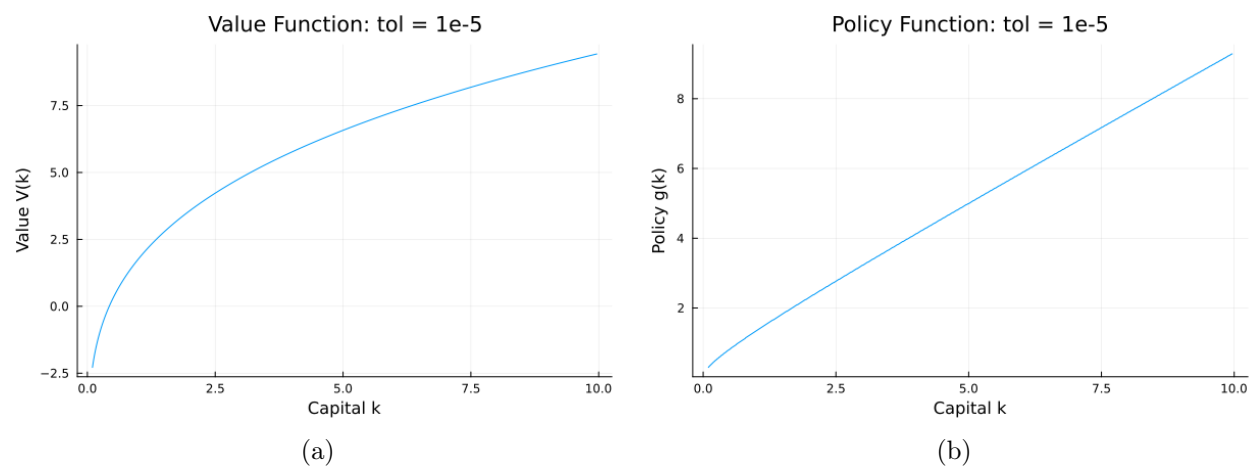


Figure 3: $V(k)$ and $g(k)$ with a tolerance of 10^{-5}

```

    out2 = standard_vfi(;N=1001,k_low=0.02, k_high=2.0, tol=1e-6)
@show out2.it out2.kstar out2.kmin out2.kmax out2.g out2.converged out2.V

plot(
    out2.kgrid,
    out2.V,
    xlabel = "Capital k",
    ylabel = "Value V(k)",
    title = "Value Function: tol = 1e-6",
    legend = false,
    grid = true
)
plot(
    out2.kgrid,
    out2.g,
    xlabel = "Capital k",
    ylabel = "Policy g(k)",
    title = "Policy Function: tol = 1e-6",
    legend = false,
    grid = true
)

out3 = standard_vfi(;N=1001,k_low=0.02, k_high=2.0, tol=1e-5)
@show out3.it out3.kstar out3.kmin out3.kmax out3.g out3.converged out3.V

plot(
    out3.kgrid,
    out3.V,
    xlabel = "Capital k",
    ylabel = "Value V(k)",
    title = "Value Function: tol = 1e-5",
    legend = false,
    grid = true
)
plot(
    out3.kgrid,
    out3.g,
    xlabel = "Capital k",
    ylabel = "Policy g(k)",
    title = "Policy Function: tol = 1e-5",
    legend = false,
    grid = true
)

```

(c) Repeat part (a) for the case of full depreciation ($\delta = 1$). Answer the same questions as in part a

- Compare the value function and decision rule you obtain numerically here to the analytical (closed-form) solutions we found in class. Plot both of them **and comment**.
- Make a table that shows the iteration numbers for all cases you solved from (a) to (d).

i) We find that the numeric solution to the value and policy function in the full depreciation case perfectly matches the analytic solution to the same problem, which is exactly what we expect.

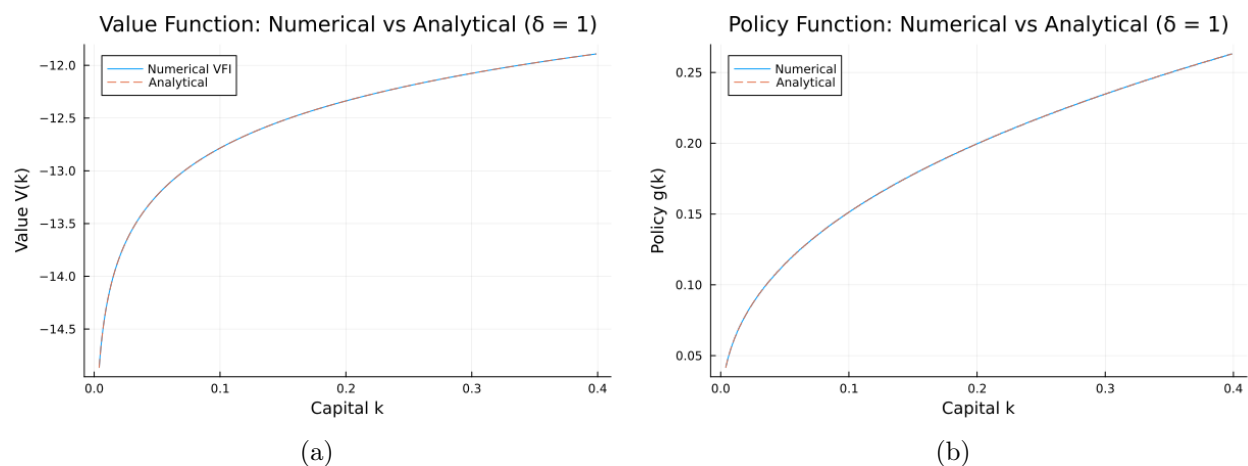


Figure 4: Numerical vs analytical solutions to $V(k)$ and $g(k)$

```

out4 = standard_vfi(N=1001,k_low=0.02, k_high=2.0, delta = 1)
@show out4.it out4.kstar out4.kmin out4.kmax out4.g out4.converged out4.V out4.
policy_idx

β = 0.95
α = 0.4
αβ = α*β
A = 1.0

a = (1/(1-β)) * (1/(1-αβ)) * (log(A) + (1-αβ)*log(αβ) + (αβ*log(αβ)))
b = α / (1 - αβ)

V_analytical = a .+ b .* log.(out4.kgrid)

V_analytical .+= out4.V[1] - V_analytical[1]

plot(out4.kgrid,
      out4.V,
      label = "Numerical VFI",
      xlabel = "Capital k",
      ylabel = "Value V(k)",
      title = "Value Function: Numerical vs Analytical (δ = 1)")

plot!(out4.kgrid,

```

```

V_analytical,
linestyle = :dash,
label = "Analytical")

plot(out4.kgrid, out4.g, label="Numerical",
xlabel = "Capital k",
ylabel = "Policy g(k)",
title = "Policy Function: Numerical vs Analytical ( $\delta = 1$ )")

plot!(out4.kgrid, 0.4*0.95*1 .* out4.kgrid.^0.4, linestyle=:dash, label="
Analytical")

```

ii) Below is a table that shows the iteration numbers for all cases we solved from (a) to (d)

Table 2: Number of VFI iterations

$\text{tol} = 10^{-8}$	$\text{tol} = 10^{-8}$	$\text{tol} = 10^{-8}$	$\text{tol} = 10^{-8}, \delta = 1$
57	57	57	16